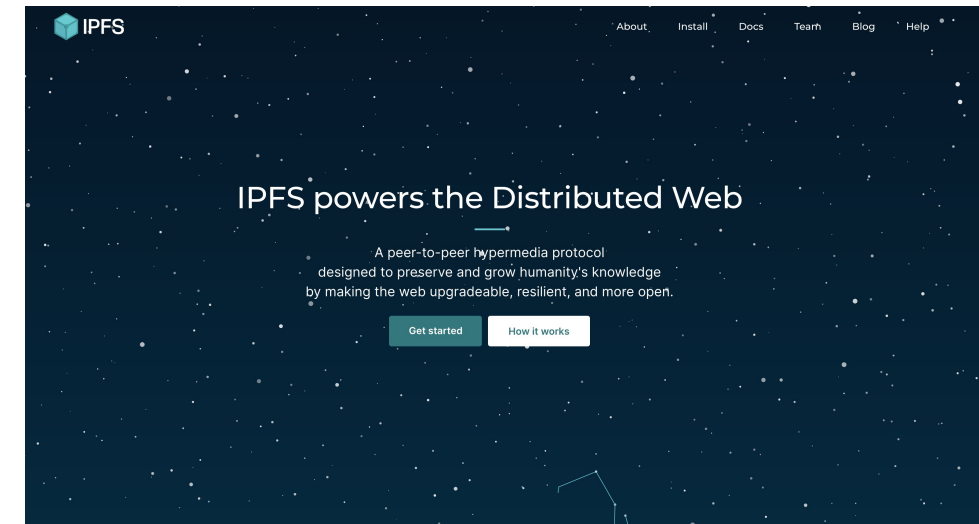
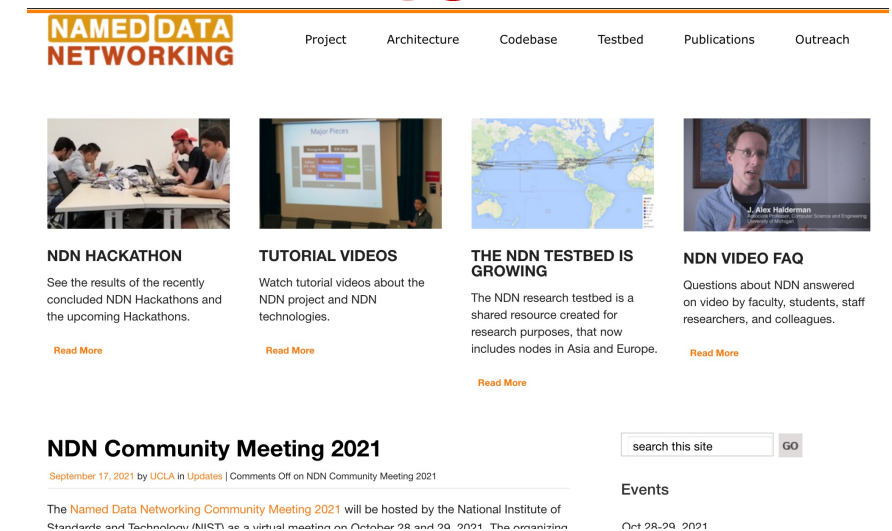


Must-haves for NDN

- **QuickStart guide**
- **Single High-level (name-level?) Library**
 - **data-oriented**, async, manifests NDN principles
 - generalized namespaces by default (objects and streams), no interest/packet exchange
 - handle cert management
- **Easy Bootstrap**
 - I just need to **move data** between my nodes
 - where is **NDN-Zeroconf**?
- **Demo Examples**
 - multi-homing (redundant storage)
 - edge services (live video ML processing)



VS



NDN FAQ does not answer the most frequently asked “**Where do I start?**” ...

Data-Oriented Networking needs

Data-Oriented Mindsets

NDNcomm 2021

Peter Gusev

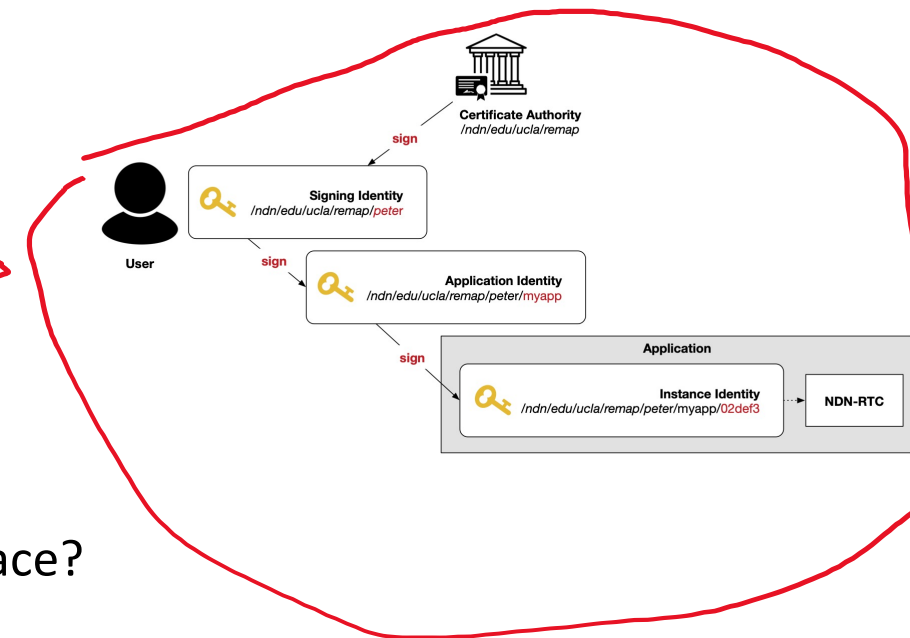
Data-Oriented Networking needs Data-Oriented *Mindsets*

- how *reacting to a stream of objects* might look in NDN:

```
Namespace stream("/touchndn/livecam/mlproc", face);  
GeneralizedObjectStreamHandler(&stream, PipelineType::Cubic,  
[&] (int seqNo, auto meta, auto obj)  
{  
    cout << "annotation " << seqNo  
        << " type " << meta->getContentType()  
        << " contents: "  
        << nlohmann::json::parse(obj.getBlobObject().toRawStr()) << endl;  
}).objectNeeded();
```

NDN Principles are not reflected/hard to follow in libraries

- #2 Data-Centricity and Data Immutability
 - why my library allows me to **publish two data** packets with the **same name**?
 - why shall I still care about using IP addresses (tcp://192.168.0.1:6363) to make my app work?
- #3 Securing Data Directly
 - how shall I manage my certificates?
 - who's handling my certs?
 - am I responsible for always-on cert repo, cert rotation, deployment...?
 - app best practices? is this the way to *"do it right"*?
- #4 Hierarchical Naming
 - cool, but... I just have a JSON to move;
 - any working namespace examples?
 - what happens to my app/data when I update the namespace?
 - how do I discover data?



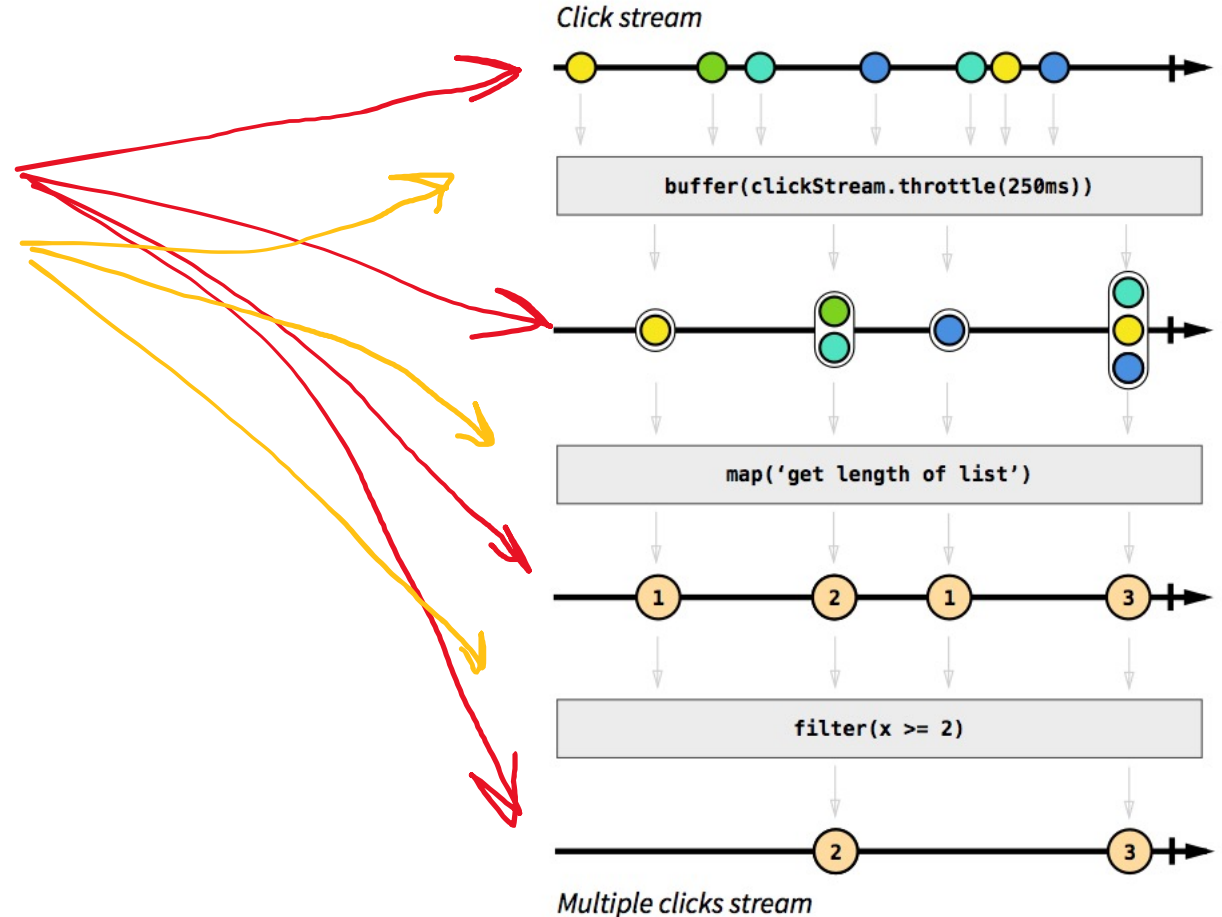
Data-Oriented Networking *needs* Data-Oriented Thinking

- *The purpose of all programs, and all parts of those programs, is to **transform data** from one form to another.*
 - from C++ “Data-Oriented design principles”, M. Acton, CppCon 2014¹
- data-oriented design is not new:
 - Model-View-Controller
 - Entity-Component-System
 - Reactive Programming
- DO app should avoid:
 - **data mutation**
 - coupling code with data

1. <https://www.youtube.com/watch?v=rX0ltVEVjHc>

Data-Oriented Networking *needs* Data-Oriented Programming

- e.g. Reactive Programming²
 - everything is a stream (of events)
 - transforms can output streams
 - publisher-subscriber paradigm
 - asynchronous
 - no runloops, use futures!



2. <https://gist.github.com/staltz/868e7e9bc2a7b8c1f754>

Data-Oriented NDN Library

- asynchronous and data-centric

```
"/myprefix/mydata"_ndn.fetch([](auto data){ cout << "got " << data.raw(); });
```

- should support common use-cases (i.e. move a JSON, versioned, multi-segmented, etc.)
- ...but in fact, should provide more (at no cost):
 - if it's a video stream – I can throttle and/or access data per Keyframe, per ROI, per GOP, etc.
 - if I have N nodes and one publisher goes offline – I can still access old data
 - if I deploy N nodes, I can setup simplest web of trust (or PKI) between them
 - if I deploy N nodes, they discover each other “automagically”